

PROJECT REPORT

TERMINAL CURRENCY & INFLATION CALCULATOR

A Python-Based Currency Conversion and Inflation Projection System

NAME	AARADHYA TIWARI
REGISTRATION NUMBER	26BAI10672
COURSE / BRANCH	B.Tech AI/ML
PROJECT TYPE	Python Programming Project
INTERFACE	Terminal / Command Line

Project Scope

This report documents the supplied Python implementation, including embedded exchange rates, monthly inflation data, currency-name mapping, input validation, USD-based conversion, inflation projection, exception handling, testing, limitations, and future scope.

CERTIFICATE

This is to certify that **AARADHYA TIWARI**, Registration Number **26BAI10672**, a student of **B.Tech AI/ML**, has prepared the project report entitled “**Terminal Currency & Inflation Calculator**”. The report presents the design and implementation of a Python-based terminal application for currency conversion and optional inflation projection using embedded datasets.

Student Signature	Faculty / Project Guide
AARADHYA TIWARI	_____
Date: _____	Date: _____

DECLARATION

I, **AARADHYA TIWARI**, hereby declare that this project report titled “**Terminal Currency & Inflation Calculator**” describes the Python project submitted by me for academic purposes. The report explains the problem, objectives, requirements, design, implementation, testing, limitations, and future scope of the supplied program.

The project uses embedded data for exchange rates, monthly inflation rates, and selected currency names. The report documents the implementation as supplied and does not represent the embedded values as live market data.

Name	AARADHYA TIWARI
Registration Number	26BAI10672
Course / Branch	B.Tech AI/ML
Signature	_____
Date	_____

ABSTRACT

The **Terminal Currency & Inflation Calculator** is a Python-based command-line application designed to perform currency conversion and optional inflation-adjusted value projection. The program stores exchange rates in a Python dictionary using USD as the base currency. It also stores monthly inflation rates and a mapping of selected currency codes to their full names.

The application accepts a source currency, target currency, amount, and number of months. Input is validated before calculations are performed. Currency conversion takes place in two stages: the source amount is first converted to USD and then converted from USD to the target currency. When an inflation projection is requested and data exists, the program applies a compound monthly formula.

The project demonstrates dictionaries, functions, loops, conditions, string methods, exception handling, floating-point arithmetic, formatted output, and modular program organization. Because the data is embedded in the source code, the application can run without an external API. However, embedded rates may become outdated.

Keywords

Python • Currency Converter • Inflation Projection • Dictionaries • Functions • Input Validation • USD Base Rate • Terminal Application • Financial Calculation

TABLE OF CONTENTS

No.	Section
1	Introduction
2	Problem Statement
3	Aim and Objectives
4	Existing and Proposed System
5	Scope
6	Functional Requirements
7	Non-Functional Requirements
8	Technology and Tools
9	System Architecture
10	Module Description
11	Algorithm and Mathematical Model
12	Data Flow and Workflow
13	Implementation Details
14	User Interface and Output Design
15	Testing and Test Cases
16	Complexity and Performance
17	Security and Reliability
18	Limitations
19	Future Enhancements
20	Conclusion
21	References / Project Basis

1. INTRODUCTION

Currency conversion is a common computational task in international travel, education, e-commerce, business, and financial planning. Different countries use different currencies and the relative value between currencies changes over time. A software calculator can automate the arithmetic involved in converting an amount from one currency to another.

This project extends currency conversion with an optional inflation projection. After converting an amount into the target currency, the program can apply the target currency's stored monthly inflation rate for a user-specified number of months. This provides a mathematical projection based on the supplied rate.

The program contains three principal data structures: **EXCHANGE_RATES**, **INFLATION_RATES**, and **CURRENCY_NAMES**. Two helper functions validate input and **main()** coordinates the complete application.

2. PROBLEM STATEMENT

Formal Problem Statement

Develop a Python terminal application that accepts a source currency, target currency, amount, and number of months; validates the input; converts the amount through a USD-based exchange-rate system; and optionally calculates an inflation-adjusted future value using the stored monthly inflation rate of the target currency.

Manual currency conversion requires a user to find rates and perform calculations. Repeating the process for different currencies can be inconvenient and can introduce arithmetic or input errors. The proposed application stores the rates in structured dictionaries and automates the calculation.

The optional inflation component addresses a second computational need: showing how a converted amount changes mathematically when one fixed monthly inflation rate is compounded over a selected number of months.

3. AIM AND OBJECTIVES

Aim: To develop a simple Python terminal application that performs currency conversion and an optional inflation-adjusted projection using embedded datasets.

Objectives

1. Store exchange-rate information in a Python dictionary.
2. Use USD as a common intermediate/base currency.
3. Accept and validate source and target currency codes.
4. Accept and validate a non-negative amount.
5. Display converted values clearly.
6. Display currency names when available.
7. Accept a projection period in months.
8. Calculate a compound inflation projection when data exists.
9. Handle missing inflation data without crashing.
10. Handle Ctrl+C using KeyboardInterrupt.
11. Demonstrate fundamental Python programming concepts.

4. EXISTING SYSTEM AND PROPOSED SYSTEM

Aspect	Manual Approach	Proposed Python Application
Exchange rate lookup	Manual / external search	Embedded dictionary
Conversion	Manual calculation	Automatic
Validation	User responsibility	Program validation
Inflation projection	Separate calculation	Built-in optional module
Internet dependency	May be required	Not required for embedded data
Repeatability	Manual	Same algorithm each time

5. SCOPE OF THE PROJECT

In scope: terminal interaction, embedded exchange rates, embedded inflation rates, USD-based conversion, non-negative amount validation, optional inflation projection, currency-name display, and error handling.

Out of scope: live market rates, live inflation feeds, graphical interface, database storage, user accounts, payment processing, money transfers, and guaranteed economic forecasting.

6. FUNCTIONAL REQUIREMENTS

ID	Requirement	Implementation
FR-01	Display application information	Header and data-date notice
FR-02	Accept source currency	get_currency_code()
FR-03	Validate source currency	EXCHANGE_RATES membership
FR-04	Accept target currency	get_currency_code()
FR-05	Validate target currency	EXCHANGE_RATES membership
FR-06	Accept amount	get_positive_number()
FR-07	Reject negative amount	val >= 0
FR-08	Handle invalid numeric input	ValueError
FR-09	Accept projection period	Month input
FR-10	Convert currency	USD intermediate calculation
FR-11	Display result	Formatted output
FR-12	Project inflation	Compound formula
FR-13	Handle missing inflation	None check
FR-14	Handle Ctrl+C	KeyboardInterrupt

7. NON-FUNCTIONAL REQUIREMENTS

Category	Requirement
Usability	Clear prompts and readable terminal output.
Performance	Fast dictionary lookup and arithmetic for normal use.
Reliability	Invalid input should be handled without breaking validation loops.
Maintainability	Functions and dictionaries make the code easier to modify.
Portability	Standard Python features; no third-party packages required.
Availability	Embedded data permits offline execution.
Accuracy	Stored rates are applied consistently using the defined formulas.
Readability	Headings, separators, labels, and formatted numbers.

Category	Requirement
Extensibility	Can later support APIs, GUI, databases, and reports.

8. TECHNOLOGY AND TOOLS

Component	Technology
Programming language	Python
Interface	Terminal / Command Line
Data storage	Python dictionaries
Database	None
External API	None
Third-party packages	None required
Core concepts	Functions, loops, conditions, dictionaries, exceptions

Example execution command:

```
python currency_calculator.py
```

9. SYSTEM ARCHITECTURE

The system uses a simple functional architecture in which user input passes through validation and then into the calculation logic. The calculation logic reads the embedded datasets and sends formatted results back to the terminal.

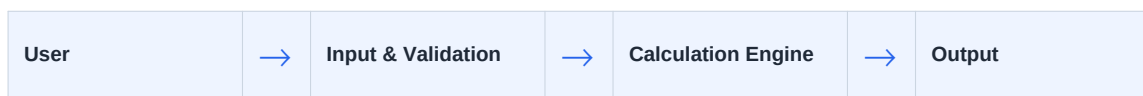


Figure 1. High-level architecture.

Layer	Responsibility
User / Terminal	Receives input and displays results.
Validation	Checks currency codes and numerical values.
Data	Provides exchange rates, inflation rates, and names.
Calculation	Performs USD conversion and inflation projection.
Output	Formats results and warnings.

10. MODULE DESCRIPTION

10.1 EXCHANGE_RATES

The **EXCHANGE_RATES** dictionary stores rates relative to USD. For example, **INR: 95.88** is interpreted by the program as 1 USD = 95.88 INR.

10.2 INFLATION_RATES

The **INFLATION_RATES** dictionary stores monthly inflation percentages. **None** indicates that projection data is unavailable.

10.3 CURRENCY_NAMES

The **CURRENCY_NAMES** dictionary maps selected currency codes to readable names. The program falls back to the code if a full name is not present.

10.4 get_currency_code()

This function strips whitespace, converts input to uppercase, checks the code against EXCHANGE_RATES, and repeatedly asks again for invalid input.

```
def get_currency_code(prompt_text):
    while True:
        code = input(prompt_text).strip().upper()
        if code in EXCHANGE_RATES:
            return code
        print("Currency code not recognized.")
```

10.5 get_positive_number()

This function converts input to float, accepts values greater than or equal to zero, rejects negative values, and catches ValueError.

```
def get_positive_number(prompt_text):
    while True:
        try:
            val = float(input(prompt_text).strip())
            if val >= 0:
                return val
            print("Please enter a positive number.")
        except ValueError:
            print("Invalid input.")
```

10.6 main()

The main function displays the interface, collects the four inputs, performs conversion, optionally performs inflation projection, and prints the results.

11. ALGORITHM AND MATHEMATICAL MODEL

Currency conversion uses USD as a common base instead of storing every possible currency pair.

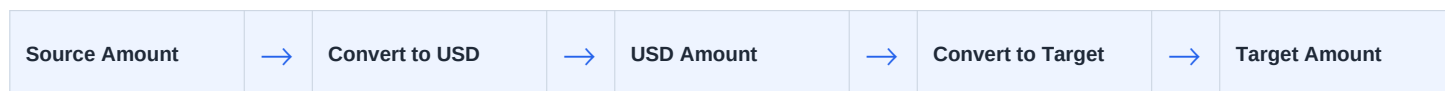


Figure 2. Currency conversion path.

Step 1:

```
amount_in_usd = amount / EXCHANGE_RATES[from_code]
```

Step 2:

```
converted_amount = amount_in_usd * EXCHANGE_RATES[to_code]
```

Combined formula:

$$\text{Target Amount} = \text{Source Amount} \times (\text{Target USD Rate} / \text{Source USD Rate})$$

For inflation projection, when months > 0 and data exists:

$$\text{Future Value} = \text{Current Value} \times (1 + \text{inflation_rate} / 100) ^ \text{months}$$

Example for a converted value of 9,588, monthly inflation of 0.4%, and six months:

$$\text{Future Value} = 9588 \times (1 + 0.004)^6$$

This is a constant-rate mathematical projection, not a guarantee of actual future inflation.

12. DATA FLOW AND WORKFLOW

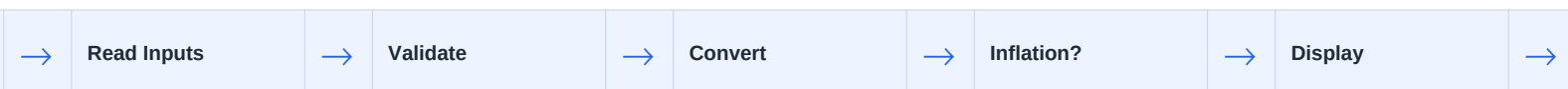


Figure 3. End-to-end workflow.

1. Display title and data-date notice.
2. Read and validate the source currency.

3. Read and validate the target currency.
4. Read and validate the amount.
5. Read the number of months.
6. Convert source amount to USD.
7. Convert USD to target currency.
8. Display the conversion result.
9. If months > 0, retrieve target inflation data.
10. If available, calculate the future value.
11. Otherwise display the missing-data warning.
12. Finish normally or handle Ctrl+C.

13. IMPLEMENTATION DETAILS

The program is self-contained: all three major datasets are written directly into the source file. This simplifies execution but requires manual updates when new values are needed.

Input normalization is performed using:

```
code = input(prompt_text).strip().upper()
```

This allows entries such as **usd**, **USD**, and **Usd** to be treated as **USD**.

The outer exception handler catches Ctrl+C:

```
try:
    main()
except KeyboardInterrupt:
    print("\n\nCalculator closed.")
```

The program also uses **CURRENCY_NAMES.get(code, code)**, which provides a safe fallback when a full currency name is not mapped.

The supplied code prints a notice stating that exchange rates are based on September 20, 2026 data and that the current date is September 22, 2026. These are program-provided labels; the report does not independently verify the embedded rates.

14. USER INTERFACE AND OUTPUT DESIGN

```
=====
■ Terminal Currency & Inflation Calculator ■
-----
Notice: Exchange rates are based on September 20, 2026 data.
Current date: September 22, 2026.
=====
```

Conversion output is presented using the source and target amounts and readable currency names when available.

```
=====
■ CONVERSION RESULT
=====
From : 100.00 USD (United States dollar)
To   : 9,588.00 INR (Indian rupee)
```

When inflation is requested, a separate projection section is displayed.

15. TESTING AND TEST CASES

ID	Scenario	Expected Result
T01	USD → INR, 100	Conversion displayed

ID	Scenario	Expected Result
T02	INR → USD	Reverse conversion displayed
T03	Lowercase usd	Accepted as USD
T04	Unknown XYZ	Error and retry
T05	Amount 100.50	Accepted
T06	Amount 0	Accepted
T07	Amount -50	Rejected
T08	Amount abc	ValueError handled
T09	Months 0	Projection skipped
T10	Months 6 with data	Projection calculated
T11	Months > 0 without data	Warning displayed
T12	Ctrl+C	Graceful exit

Important implementation observation: the function named `get_positive_number()` accepts zero because it checks `val >= 0`. Also, the month input is passed through `int()`, so a decimal such as 2.5 becomes 2 rather than being explicitly rejected. These are suitable points for future refinement.

16. COMPLEXITY AND PERFORMANCE

Operation	Average Complexity
Currency validation	O(1)
Exchange-rate lookup	O(1)
Inflation-rate lookup	O(1)
Currency-name lookup	O(1)
Conversion	O(1)
Inflation calculation	O(1)
Stored data	O(n) memory

Performance

The calculator is computationally lightweight. Dictionary-based lookups and arithmetic operations complete rapidly for normal use. Data freshness is a more important practical limitation than processing time.

17. SECURITY AND RELIABILITY

Security: The current application has a small external attack surface because it has no network connection, database, payment system, or authentication. User input is treated as data rather than executable Python code.

Reliability: Currency validation, numeric validation, explicit missing-inflation handling, and KeyboardInterrupt handling reduce common runtime failures.

If a live API is introduced later, API keys should be stored outside source code and external responses should be validated before use.

18. LIMITATIONS

Limitation	Impact
Embedded exchange rates	Do not automatically update.

Limitation	Impact
Embedded inflation rates	May become outdated.
No live API	No real-time market information.
Partial name mapping	Some currencies may display only their code.
Terminal interface	No GUI.
No persistence	Results are not automatically saved.
Constant inflation assumption	Projection does not model changing rates.
Month conversion	Decimal input can be truncated by int().
Floating-point arithmetic	Not designed for financial settlement-grade precision.

The application should therefore be treated as an educational calculation tool rather than a live financial service.

19. FUTURE ENHANCEMENTS

1. Live exchange-rate API integration.
2. Live inflation-data integration.
3. Move embedded data to JSON, CSV, or a database.
4. Graphical interface using Tkinter or another GUI framework.
5. Currency search by code and full name.
6. Currency symbols and improved formatting.
7. Conversion history and saved results.
8. CSV/PDF export.
9. Automated unit testing.
10. Historical exchange-rate and inflation graphs.
11. Strict integer validation for months.
12. Decimal-based monetary calculations where higher precision is required.
13. Logging and robust API/network error handling.
14. Separate modules for data, validation, conversion, inflation, display, and testing.

20. CONCLUSION

The Terminal Currency & Inflation Calculator is a practical Python project that combines currency conversion with an optional inflation-adjusted projection. It demonstrates dictionaries, functions, loops, conditions, string processing, exception handling, arithmetic operations, and formatted terminal output.

The use of USD as a common base simplifies conversion between multiple currencies. The inflation module uses a compound monthly formula and explicitly handles unavailable inflation data. Input validation and keyboard-interruption handling improve usability and reliability.

The project is suitable as a first-year programming project and provides a foundation for future development into an API-based, graphical, database-backed, or reporting-oriented financial calculation application.

Student / Project Details

Name: AARADHYA TIWARI

Registration Number: 26BAI10672

Course / Branch: B.Tech AI/ML

Project: Terminal Currency & Inflation Calculator

21. REFERENCES / PROJECT BASIS

This report is based directly on the Python source code supplied for the project. Implementation-specific descriptions, function names, formulas, data structures, validation behavior, and limitations are derived from that supplied code.

The embedded exchange-rate and inflation values are treated as project data exactly as represented in the supplied source. They are not independently verified or presented as live financial data in this report.

— END OF PROJECT REPORT —